

# Open Science in Practice: Collaborative Research with Git



Niklas Edvall, PhD  
niklas.edvall@ki.se

K8F6106  
HT2026



Andreas Gerhardsson, PhD  
andreas.Gerhardsson@ki.se

# Git remote

Andreas Gerhardsson

# Your local Git



Git course



.git

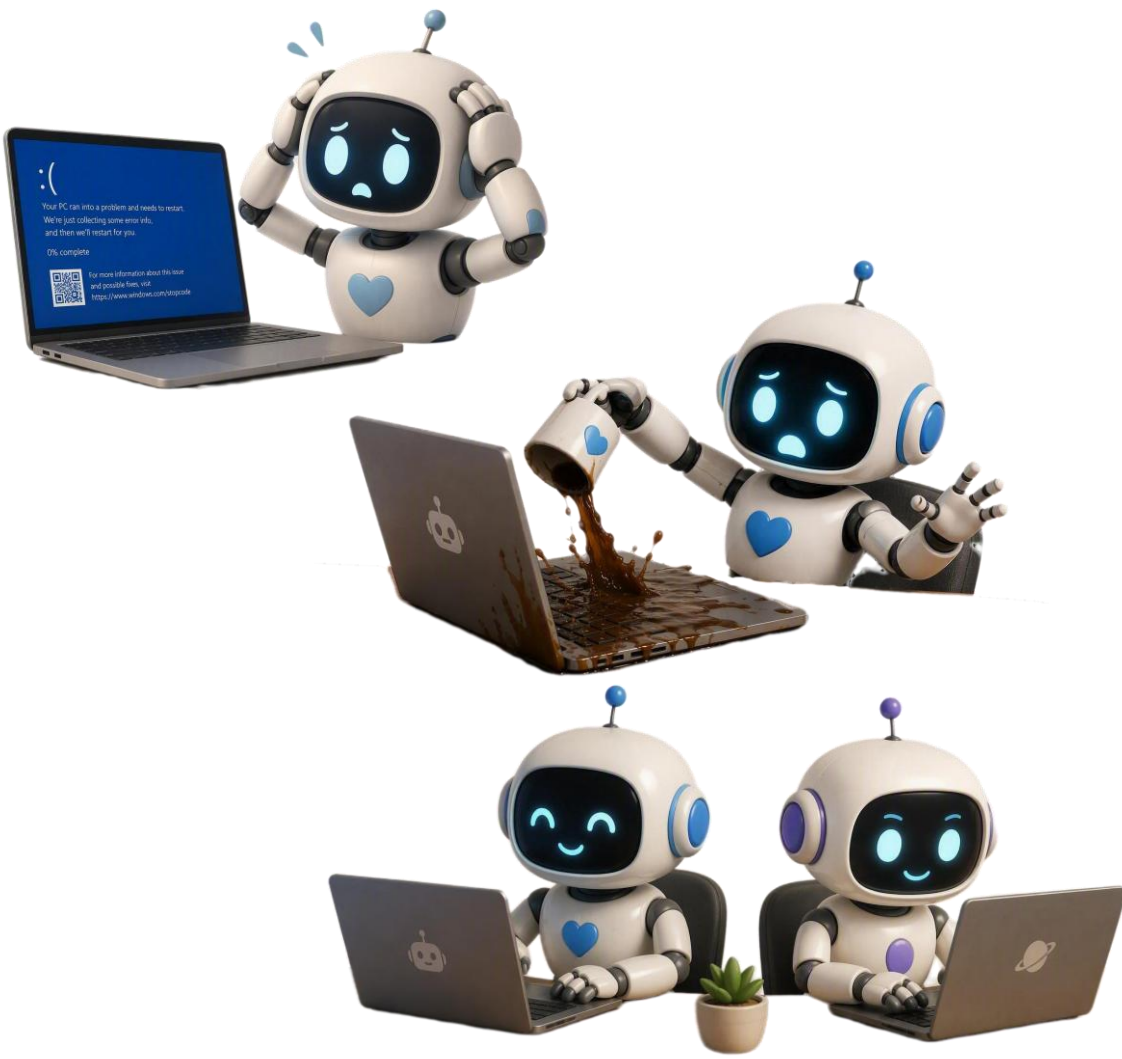


read\_data.R



run\_models.R

# What if?



# A remote repository



Git course



.git



read\_data.R



run\_models.R



Git course



.git

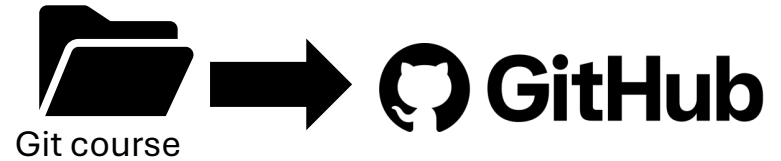


read\_data.R



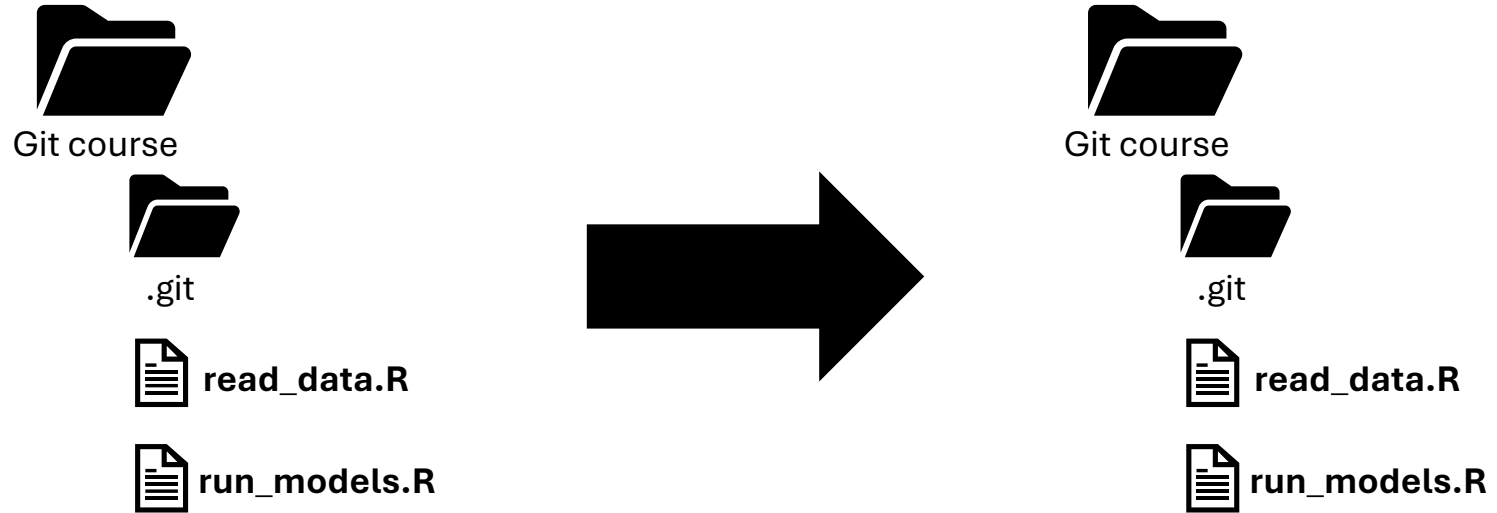
run\_models.R

# Creating a remote



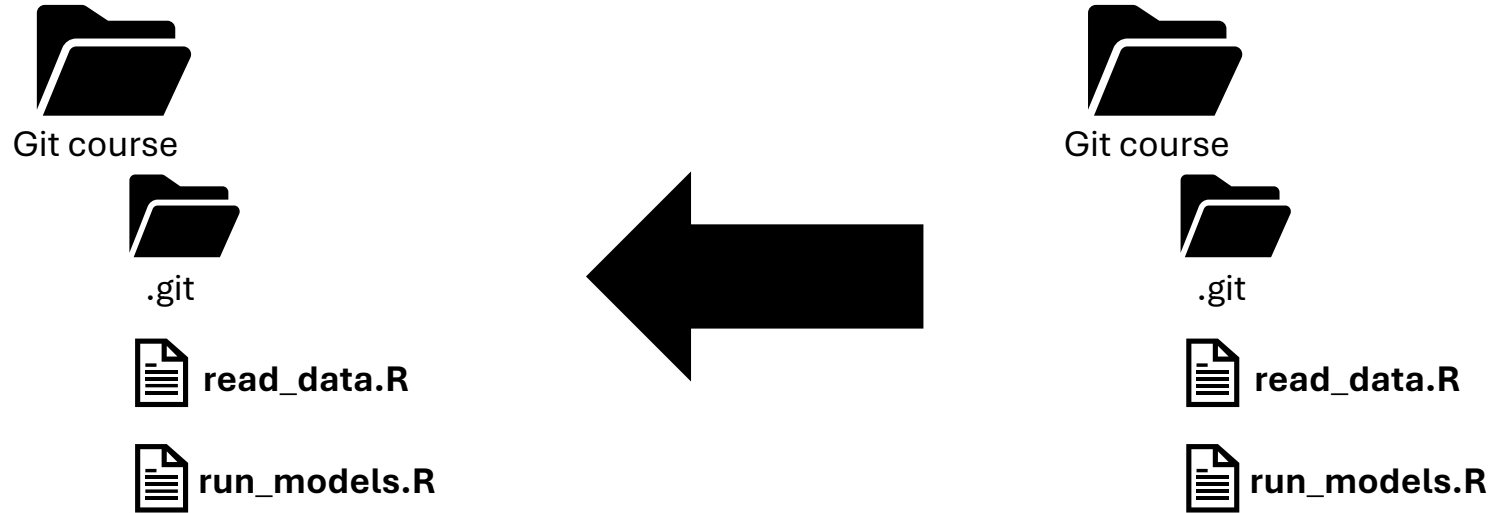
```
andreas ~ % git remote add origin \  
git@github.com:k-CIR/course-  
open-science-in-practice.git
```

# Local to remote



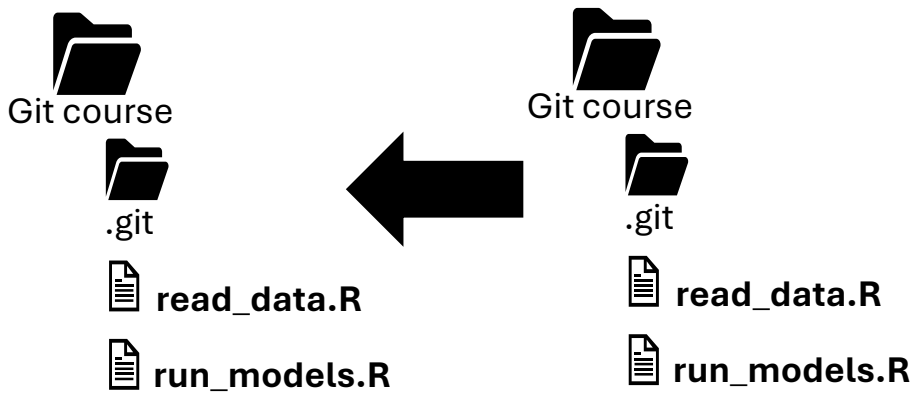
```
andreas ~ % git push
```

# Remote to local



```
andreas ~ % git pull [clone, fetch]
```

# Remote to local



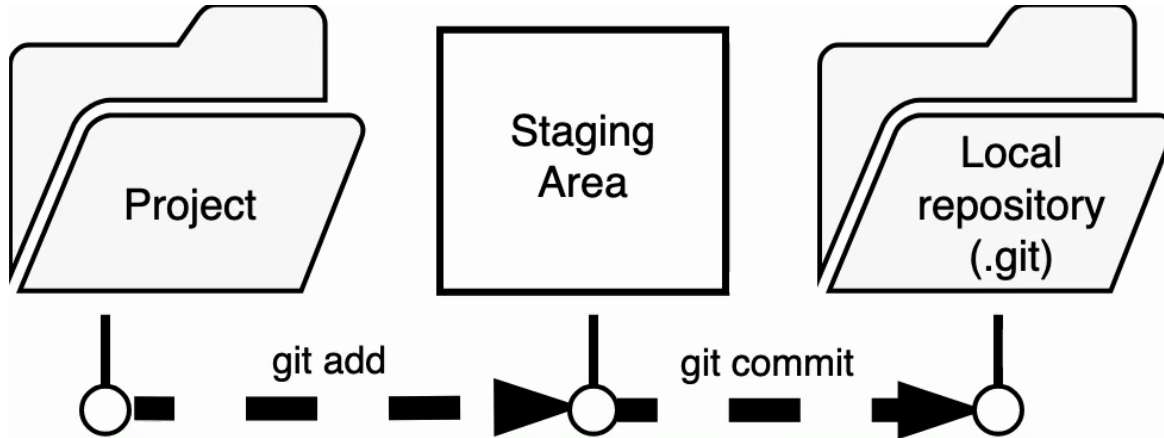
**git clone:** copy the entire repository, run once on fresh start

**git fetch:** download new commits and branches, does not touch your current work.

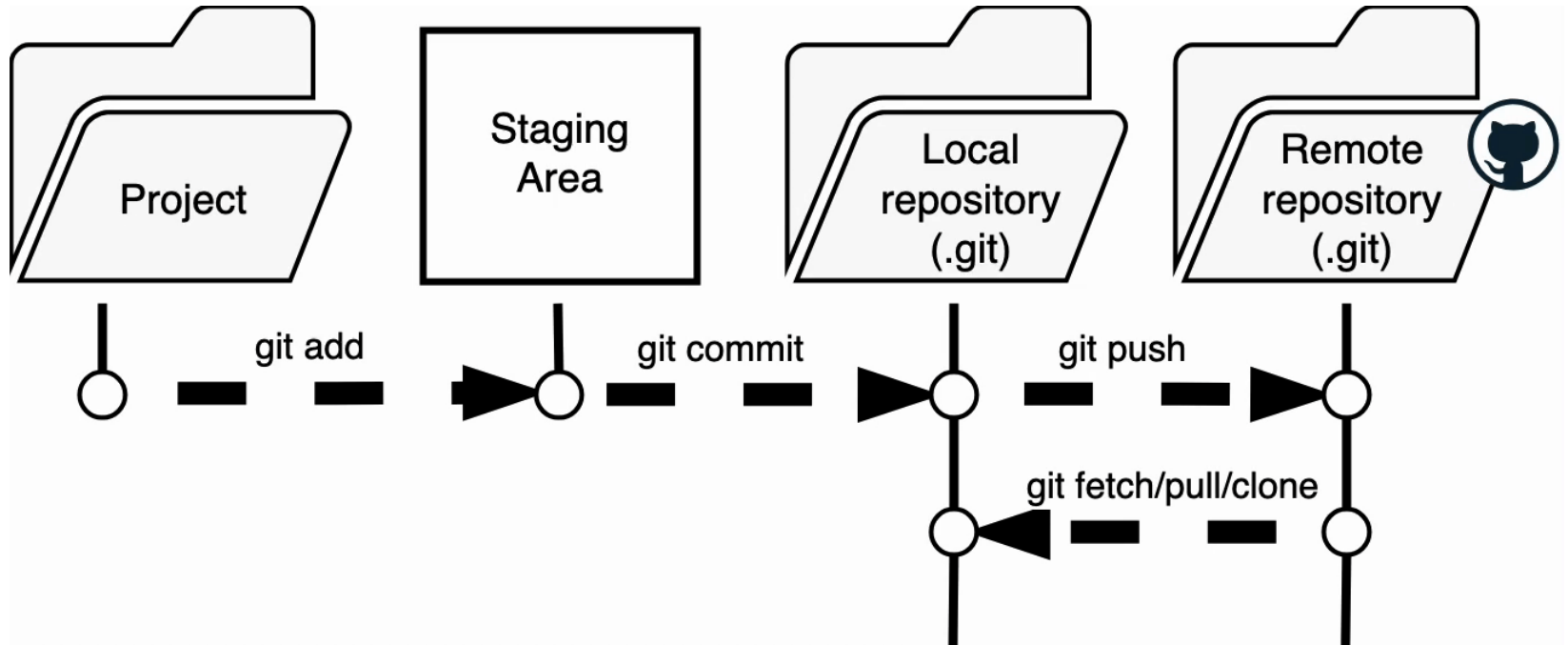
**git pull:** fetch + merge

**Sync:** not a git concept, but can be an option in some IDE (= pull then push)

# The local Git workflow



# The remote Git workflow



# What gets committed, stays committed



1. Be careful when you use ``git add`` especially ``git add .``
2. Never commit credentials, tokens, or sensitive data
3. Set up a `.gitignore` early and keep it updated

---

If you accidentally pushed something secret.

1. Delete remote repo
2. Use tools like **git-filter-repo**  
<https://github.com/newren/git-filter-repo>

# .gitignore



Git course



.git



read\_data.R



run\_models.R



.gitignore

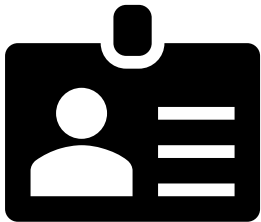
```
# Junk
.DS_Store
.Rhistory
__pycache__
```

```
# Environments
.venv
```

```
# Folders and output
data
figures/*
*.png
*.csv
```

**commit source and documentation – ignore data, outputs, and secrets**

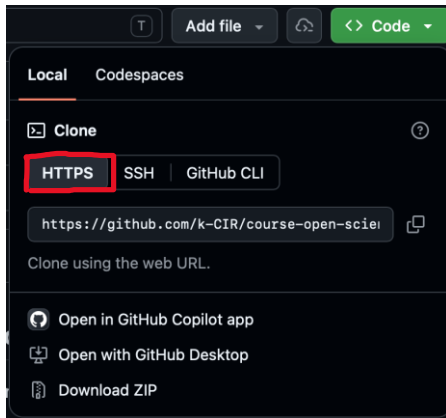
# Communication with GitHub



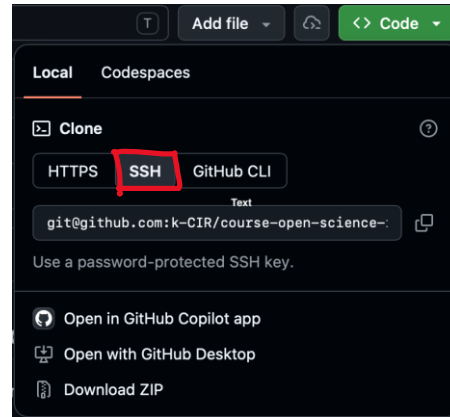
- Git tracks changes, what, when, why – **by whom**

```
andreas ~ %  
git config --global user.name "Your Name"  
git config --global user.email "you@email.com"
```

# HTTPS vs SSH



- Asks for a username and a personal access token on every push



+ provide a smoother and more secure alternative

Need a public SSH-key

# Setup SSH link

## Local computer

- Private key (never share)
  - Public key (put on GitHub)
1. Generate a key-pair
  2. Start the agent and add the key
  3. Add the **public** key to GitHub

```
andreas ~ %
```

```
ssh-keygen -t ed25519 -C "you@example.com"
```

```
eval "$(ssh-agent -s)"
```

```
ssh-add ~/.ssh/id_ed25519
```

```
cat ~/.ssh/id_ed25519.pub
```

```
ssh-ed25519
```

```
AAAAB3NzaC1yc2EAAAADAQABAAQGC7vQ2k9ZpXwM1nJf8dR4t  
L6yV0qWx3aN2bK5cM9pE1sH7gU4rT8wY6xZ2vB0nM3jK9lQpW5eR  
7tY1ul8oP0aS2dF4gH6jK8lZ1xC3vB5nM7pQ9rT2wY4ul6oP8aS0dF2  
gH4jK6lZ8xC1vB3nM5pQ7rT9wY1ul3oP5aS7dF9gH1jK3lZ5xC7vB9n  
M1pQ3rT5wY7ul9oP1aS3dF5gH7jK9lZ2xC4vB6nM8pQ0rT3wY5ul7o  
P9aS1dF3gH5jK7lZ9xC2vB4nM6pQ8rT1wY3ul5oP7aS9dF1gH3jK5lZ  
7xC9vB1nM3pQ5rT7wY9ul1oP3aS5dF7gH9jK1lZ4xC6vB8nM0pQ2rT  
4wY6ul8oP0aS2dF4gH6jK8lZ0xC you@example.com
```

# Setup SSH link



1. Copy the SSH public key to your clipboard
2. In the upper-right corner of any page on GitHub, click your profile picture, then click  **Settings**.
3. In the "Access" section of the sidebar, click  **SSH and GPG keys**.
4. Click **New SSH key** or **Add SSH key**.
5. In the "Title" field, add a descriptive label for the new key. For example, if you're using a personal laptop, you might call this key "Personal laptop".
6. Select verification type, select **Authentication Key**
7. In the "Key" field, paste your public key.
8. Click **Add SSH key**

# Setup SSH link

## Local computer

- Private key (never share)
  - Public key (put on GitHub)
1. Generate a key-pair
  2. Start the agent and add the key
  3. Add the **public** key to GitHub
  4. Test connection

```
andreas ~ %  
  
ssh-keygen -t ed25519 -C "you@example.com"  
  
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519  
  
cat ~/.ssh/id_ed25519.pub  
  
ssh -T git@github.com
```

# Summary

- A remote repository allows you to safely backup your project and share with others
- GitHub is the most common remote repository
- A remote is a URL with which you communicate
- Never commit sensitive or secret information (both local and remote)
- Use `.gitignore` to filter out files and folders you want to leave out of Git
- Create a key-pair to communicate easily with your remote

# References

Chacon, S. (with Straub, B.). (2014). *Pro Git* (2nd ed). Apress L. P.